

GitHub Repo Link:

<https://github.com/antoniokhemmoro7/Courtside-Analytics-Project>

Project Goals

The goal of our project was to explore whether NBA team performance is related to how much teams travel based on a 300 game sample from the 2024-25 NBA season. We planned to collect NBA team and game data from the NBA API, then use geographic coordinates from OpenStreetMap (Nominatim) to attach latitude and longitude to each game. We planned to store all data in a single normalized SQLite database with multiple tables, then compute team level travel distance and win percentage from the stored data. Finally, we planned to create multiple visualizations that summarize team travel, team win percentage, and the relationship between travel distance and performance.

Achieved Goals

We successfully pulled team and game data from the NBA API and stored it in a single SQLite database. We also used the OpenStreetMap Nominatim API to geocode game locations and store latitude/longitude linked to each game by game_id. We limited each data collection run to 25 items and reran the scripts multiple times to exceed 100 total stored records from each source without duplicates, as required by the project rubric. After the database was populated, we selected and joined data across tables to compute total miles traveled per team (using the Haversine formula) and team win percentage based on the games stored. We generated four visualizations: a bar chart of team travel distance, a bar chart of team win percentage, a scatter plot comparing travel distance vs win percentage with a trendline, and a conference level comparison bar chart summarizing travel for Eastern vs Western conference teams.

Problems Faced

One of the main challenges we encountered was working with multiple APIs that had different limitations and formats. The NBA API provides structured basketball data through a Python wrapper rather than raw HTTP requests, while the OpenStreetMap Nominatim API requires direct requests.get calls and stricter rate limiting. Understanding how to properly integrate data from both sources while following the project requirement of limiting each run to 25 items required careful planning and testing.

Another challenge was managing incremental data collection without introducing duplicate records. Because we were required to rerun our data collection scripts multiple times to exceed 100 stored records per API, we had to ensure that previously collected data was not inserted again. We addressed this by designing our database tables with primary keys and using INSERT OR IGNORE statements, which allowed us to safely add new rows while preventing duplicates when scripts were re-run.

We also encountered difficulties when calculating team travel distance. Game location data is not directly available from the NBA API, so we had to attach geographic coordinates to each game using the OpenStreetMap API. This required joining data across multiple tables using a shared game_id and then computing distances between consecutive games for each team. Implementing the Haversine formula correctly and ensuring games were processed in chronological order was necessary to avoid incorrect distance calculations.

Lastly, designing clear and readable visualizations was challenging due to the number of teams involved. Initial plots were cluttered and difficult to interpret. We addressed this by sorting data, using horizontal bar charts, adding color distinctions between different visualizations, and creating a summary level conference comparison chart to highlight broader trends without overwhelming the viewer.

Calculations

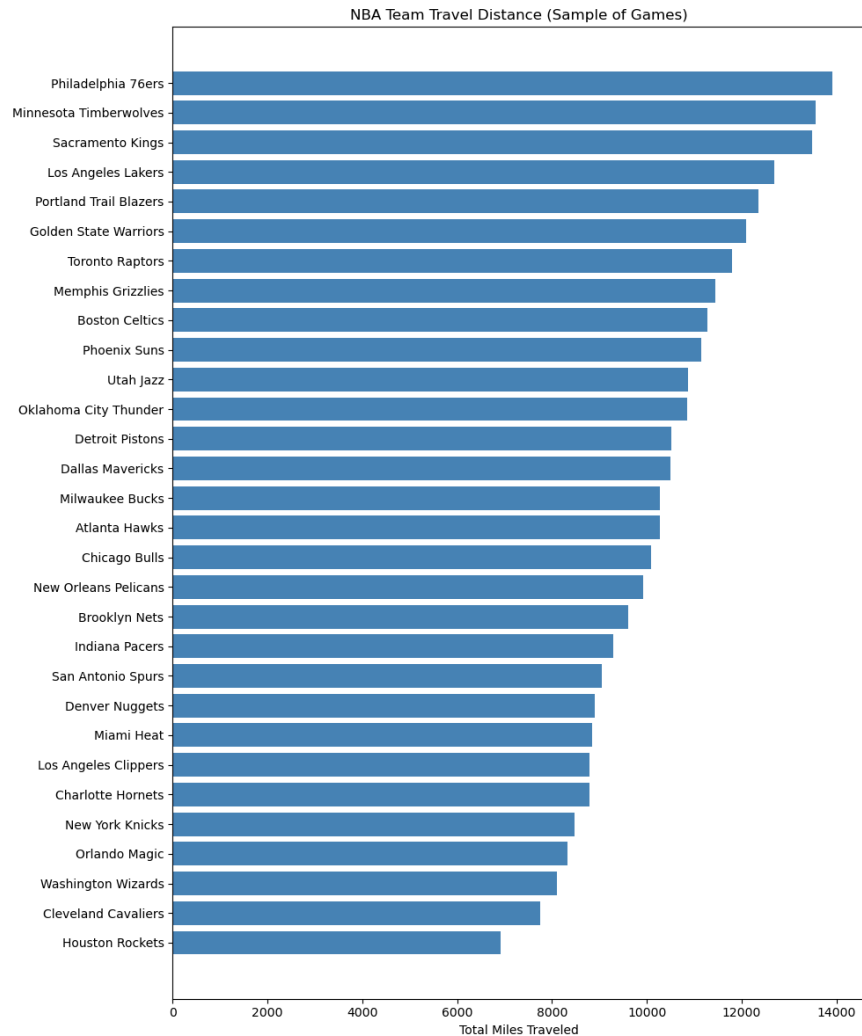
Our project required several calculations to analyze the relationship between NBA team travel and performance. All calculations were performed using data selected directly from our SQLite database after joining relevant tables.

The first major calculation was total travel distance per team. After joining the games and game_locations tables using the shared integer key game_id, we grouped games by team and sorted each team's games by date. For each team, we calculated the distance between consecutive games using the Haversine formula, which computes the great circle distance between two geographic coordinates. These distances were summed to produce a total number of miles traveled by each team over the sampled games.

The second calculation was team win percentage. Using game scores stored in the games table, we counted the total number of games played and the number of wins for each team. A win was assigned based on whether the team's score was greater than its opponent's score in a given game. Win percentage was then calculated as the number of wins divided by the total number of games played for each team. All calculations were derived from SQL SELECT statements and table joins, which ensured that no values were hard coded and that results were reproducible from the database contents alone.

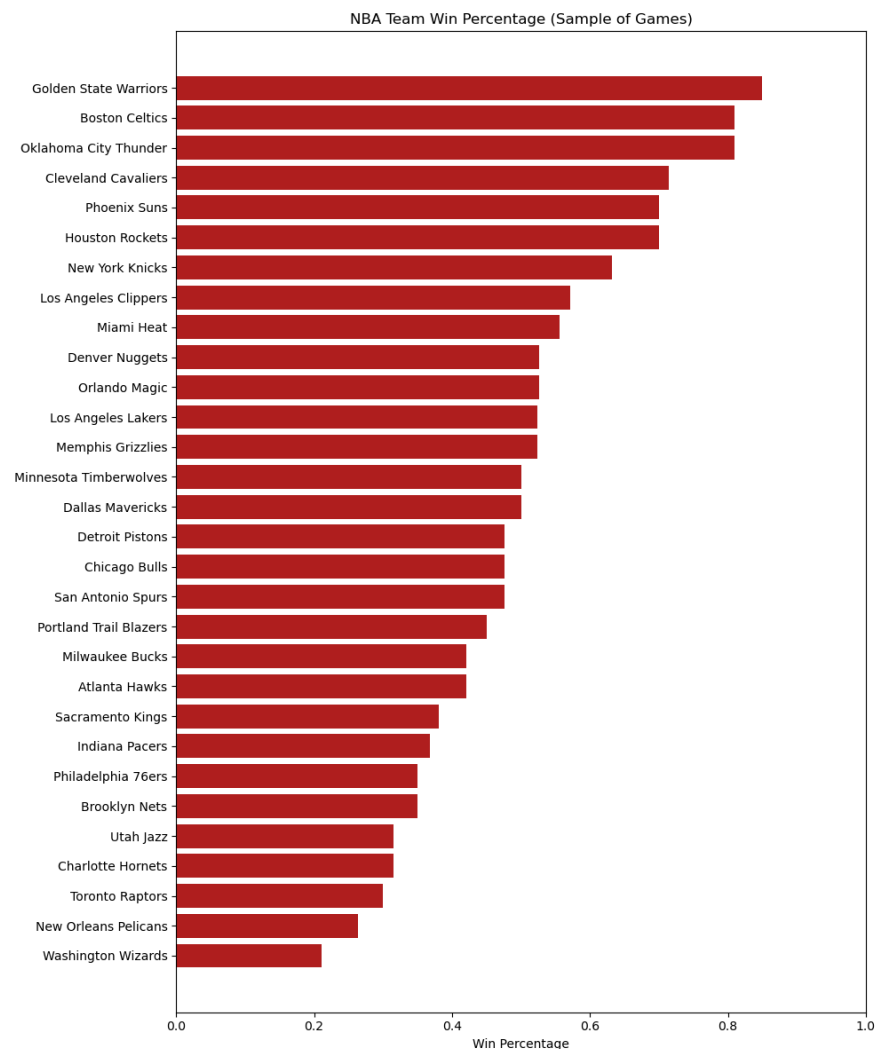
Visualizations

We created four visualizations to explore team travel patterns, team performance, and the relationship between travel distance and win percentage based on a 300 game sample from the 2024-25 NBA season. Each visualization is based on calculations derived directly from the database and is intended to answer a specific part of our research question.

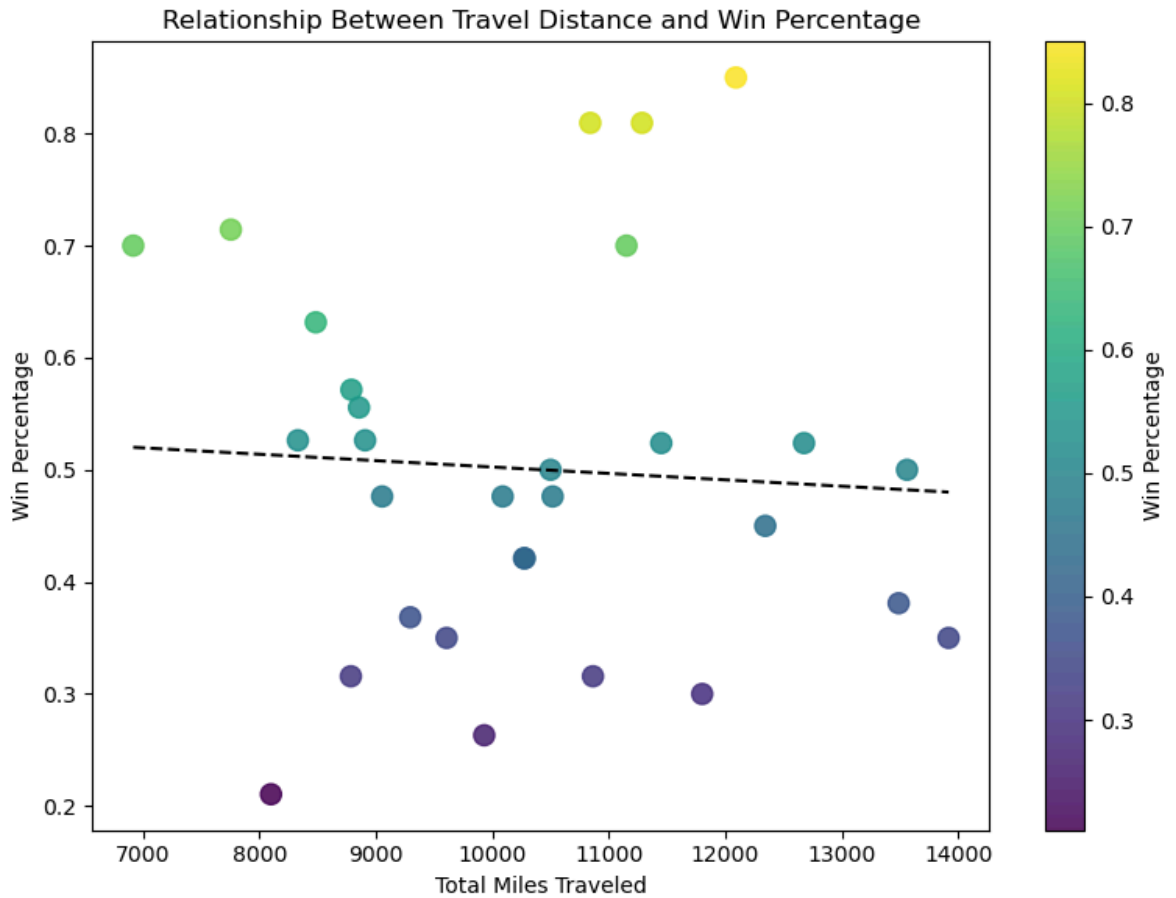


This bar chart represents the total miles traveled by each NBA team across the 300 sampled games. The chart highlights how travel distance varies substantially between teams, with some teams experiencing much higher travel demands than others. Teams such as the Philadelphia 76ers, Minnesota Timberwolves, and Sacramento Kings appear among the highest in total miles traveled, while teams like the Houston Rockets and Cleveland Cavaliers traveled

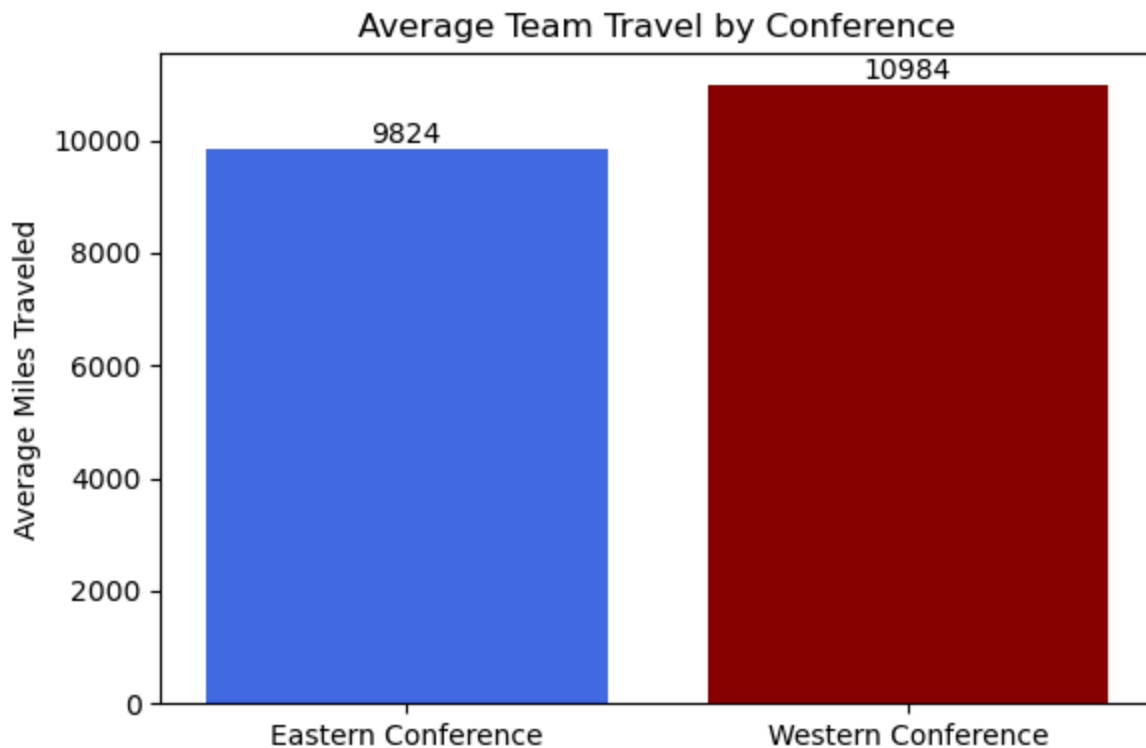
fewer miles in the same sample. This visualization provides important context for understanding differences in team travel before considering performance.



This bar chart represents team win percentage based on the same set of 300 sampled games. Win percentage is used instead of total wins to account for differences in the number of games played by each team. Teams such as the Golden State Warriors, Boston Celtics, and Oklahoma City Thunder show higher win percentages, while other teams demonstrate more moderate or lower performance. This visualization summarizes overall team success independently of travel distance.



This scatter plot represents the relationship between total travel distance and win percentage for each team. Each point corresponds to a team, with color intensity indicating win percentage and a dashed trend line highlighting the overall pattern. The plot suggests a slight negative relationship between travel distance and win percentage. However, the wide spread of points indicates that travel distance alone does not determine team performance. This visualization is the primary analysis used to explore potential correlation without implying causation.



This bar chart represents average team travel distance by conference, comparing Eastern and Western Conference teams. By aggregating travel distance at the conference level, this visualization reveals broader geographic patterns that are less visible in team level charts. Western Conference teams traveled more miles on average than Eastern Conference teams in the sampled games, reflecting the larger geographic spread of teams in the Western Conference.

Instructions to Run Code

To run this project, all files should be located in the same directory along with the SQLite database file (**nba.db**). The following steps describe the order in which the scripts should be executed to reproduce the results.

Before running the scripts, all required Python packages must be installed. This project relies on the **nba_api** library to access NBA team and game data, as well as matplotlib and numpy to perform calculations and generate visualizations. These packages must be installed using pip prior to execution. All other libraries used in the project are included in the standard Python library and do not require additional installation.

1. Fetch NBA teams

Run **fetch_nba_teams.py** to retrieve NBA team information from the NBA API and store it in the teams table. This script limits each run to 25 records and can be run

multiple times without creating duplicates.

2. Fetch NBA games

Run **fetch_nba_games.py** to retrieve NBA game data from the NBA API and store it in the games table. Each run inserts up to 25 games, and the script can be re-run multiple times to incrementally collect more games.

3. Fetch game locations

Run **fetch_game_locations.py** to retrieve latitude and longitude coordinates for game locations using the OpenStreetMap Nominatim API. This script links each location to a game using the shared game_id key and inserts up to 25 locations per run.

4. Calculate team travel distance

Run **calculate_travel.py** to compute total miles traveled by each team using the Haversine formula based on the stored game locations.

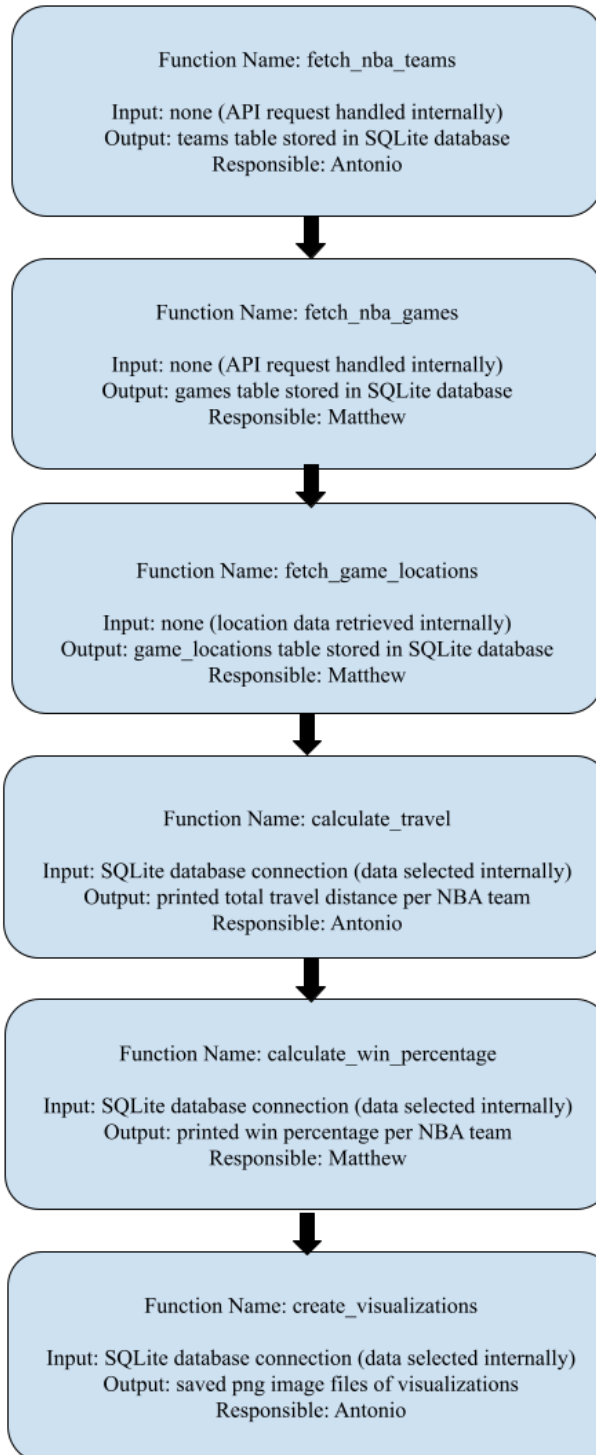
5. Calculate team win percentage

Run **calculate_win_percentage.py** to compute win percentage for each team using scores stored in the games table.

6. Create visualizations

Run **create_visualizations.py** to generate all visualizations. This script selects and joins data from the database, performs final calculations, and saves four visualization images to the project directory.

Function Diagram



Date	Issue Description	Location of Resource	Result
Dec 5, 2025	Needed to retrieve NBA team and game data programmatically	NBA API documentation and example usage	Successfully fetched NBA team and game data.
Dec 5, 2025	Needed to limit API calls to avoid pulling too much data at once	NBA API community examples	Implemented batch-based data collection.
Dec 8, 2025	Duplicate rows appeared when rerunning data collection scripts	SQLite documentation	Prevented duplicate records from being inserted.
Dec 9, 2025	Game location data was not available directly from the NBA API	OpenStreetMap Nominatim API documentation	Retrieved latitude and longitude for each game location.
Dec 11, 2025	Needed to calculate distance between consecutive game locations	Online reference explaining the Haversine formula	Calculated travel distances between locations.
Dec 11, 2025	Required combining game and location data	SQLite JOIN examples	Joined tables using shared game IDs.
Dec 12, 2025	Visualizations were cluttered and hard to read	Matplotlib documentation	Improved plot layout and readability.
Dec 12, 2025	Wanted to compare average travel by conference	Online articles discussing data aggregation	Computed average travel distances by conference.